

23. Протоколи и интерфейси.

Протоколи от транспортния слой - TCP и UDP. IP маршрутизиране. Протокол RIP.

1. Протоколи и интерфейси

1.1 Основни понятия

Мрежовият протокол е формален набор от правила, които определят синтаксиса, семантиката и последователността на съобщенията, разменяни между 2 или повече комуникиращи обекта. Протоколът отговаря на 3 въпроса:

- Какъв формат имат данните
- Кога се изпращат
- Как се реагира при грешка или загуба

Интерфейсът е граница между 2 съседни слоя в мрежовата архитектура. Той дефинира набор от операции (услуги), които долният слой предлага на горния, без да разкрива подробности за вътрешната реализация. Например транспортният слой предлага на приложния слой примитиви от типа `send()`, `receive()`, `connect()` чрез сокет интерфейса, без приложението да "знае" как точно се сегментират и препращат данните.

Разликата между протокол и интерфейс е ключова: протоколът управлява комуникацията между еднакви слоеве на различни машини (хоризонтално), докато интерфейсът управлява комуникацията между съседни слоеве на една и съща машина (вертикално).

1.2 Капсулация на данните (Data Encapsulation)

Предаването на данни от горен към долен слой се извършва чрез процеса капсулация. Всеки слой разглежда данните от горния слой като полезна информация и добавя своя служебна информация под формата на хедър (Header) и/или трейлър (Trailer).

- **Приложен слой:** Данни (Data / Message)
- **Транспортен слой:** Сегмент (Segment) / Датаграма (Datagram)
- **Мрежови слой:** Пакет (Packet)
- **Канален слой:** Рамка (Frame)
- **Физически слой:** Битове (Bits)

2. Транспортен слой

2.1 Функции на транспортния слой

Транспортният слой осигурява логическа комуникация от край до край (end-to-end) между процеси, работещи на различни хостове. Основните протоколи в TCP/IP стека са **TCP** и **UDP**, използващи концепцията за **портове** (16-битови адреси от 0 до 65535) за мултиплексиране и демултиплексиране на трафика.

2.2 TCP (Transmission Control Protocol) - Надежден, с установяване на връзка

TCP гарантира доставката на данните в същия ред, в който са изпратени, чрез управление на потока и контрол на претоварването.

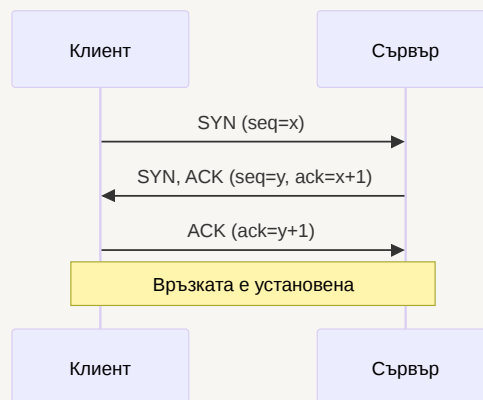
Структура на TCP сегмент

Хедърът на TCP има минимален размер от 20 байта и съдържа:

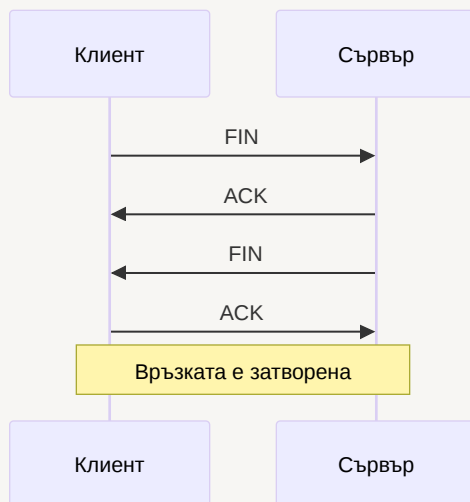
- **Source Port / Destination Port (16 бита всеки):** Идентифицират изпращащия и приемащия процес.

- **Sequence Number (32 бита):** Пореден номер на първия байт от данните в сегмента. Служи за правилно подреждане на потока.
- **Acknowledgment Number (32 бита):** Номер на следващия байт, който изпращачът очаква да получи (кумулятивно потвърждение).
- **Data Offset (4 бита):** Указва размера на TCP хедъра (на брой 32-битови думи).
- **Flags (Управляващи битове):**
 - **SYN** (Synchronize) – Инициализиране на връзка.
 - **ACK** (Acknowledgment) – Валидно поле за потвърждение.
 - **FIN** (Finish) – Прекратяване на връзката.
 - **RST** (Reset) – Насилно прекратяване/грешка.
 - **PSH** (Push) – Незабавно предаване на данните към приложното ниво.
 - **URG** (Urgent) – Спешни данни.
- **Window Size (16 бита):** Използва се за управление на потока чрез механизъм на "плъзгащия се прозорец" (*Sliding Window*). Указва броя байтове, които получателят е готов да приеме без потвърждение.
- **Checksum (16 бита):** Контролна сума за откриване на грешки в хедъра и данните.

2.2.1 Установяване на връзка



2.2.2 Прекратяване на връзка



2.2.3 Контрол на потока

TCP използва **плъзгащ се прозорец (sliding window)** - получателят обявява в полето Window Size колко байта може да приеме в момента, а подателят не изпраща повече от този обем неподтвърдени данни.

Максимален непотвърден обем $\leq$ RWND (Receiver Window)

2.2.4 Контрол на предаването

TCP поддържа отделен, вътрешен прозорец **cwnd** (congestion window), който расте постепенно, за да не претовари мрежата:

Предпазва междинната мрежова инфраструктура (рутери). Включва алгоритмите Slow Start (експоненциално нарастване на прозореца на претоварване *cwnd*), Congestion Avoidance (линейно нарастване), Fast Retransmit и Fast Recovery при откриване на загуби чрез три дублирани ACK пакета.

2.3 Протокол UDP (User Datagram Protocol)

UDP е олекотен протокол без състояние (*stateless*), който изпраща съобщения (датаграми) без гаранция за доставка, подредба или дублиране. Той има

минимален хедър от **8 байта**:

Формат на UDP хедър (8 байта общо):

Поле в UDP хедъра	Размер	Описание
Source Port	16 бита	Порт на подателя (незадължителен, може да е 0).
Destination Port	16 бита	Порт на получателя.
Length	16 бита	Дължина на целия UDP пакет (хедър + данни) в байтове.
Checksum	16 бита	Контролна сума за верификация (незадължителна при IPv4, задължителна при IPv6).

Заради ниския overhead, UDP се предпочита при приложения, чувствителни към закъснение, но толерантни към загуба на отделни пакети: VoIP, видео стрийминг на живо, DNS заявки, DHCP, RIP актуализации (виж по-долу).

2.4 Сравнение TCP и UDP

Характеристика	TCP	UDP
Установяване на връзка	Да (изисква ресурси за състояние)	Не (всяка датаграма е независима)
Надеждност	Гарантирана (преизпращане при загуба)	Липсва (Best-effort delivery)
Подредба на потока	Да (чрез Sequence Numbers)	Не (пакетите могат да пристигнат разместени)
Скорост и закъснение	По-ниска скорост, по-високо закъснение	Максимална скорост, минимално закъснение
Стрийминг/ Трансфер	Потоково ориентиран (Byte stream)	Пакето-ориентиран (Datagrams)
Типични приложения	HTTP/HTTPS, FTP, SSH, SMTP	DNS, DHCP, VoIP, Стрийминг на видео/аудио, RIP

3. IP маршрутизиране

3.1 Основна идея

Маршрутизирането е процесът на избор на оптимален път за препращане на IP пакети от източника до дестинацията през междинни мрежови устройства (рутери). Извършва се на Мрежовия слой (Слой 3) от OSI модела.

3.2 Принципи на работа на маршрутизатора (Router)

Когато рутер получи IP пакет на даден интерфейс, той изпълнява следните стъпки:

1. **Валидация:** Проверява контролната сума на Layer 2 рамката. Ако е коректна, я декапсулира, за да извлече Layer 3 IP пакета.
2. **Проверка на TTL:** Намалява стойността на полето *Time-to-Live (TTL)* в IP хедъра с 1. Ако $TTL \leq 0$, пакетът се отхвърля и се изпраща ICMP съобщение "Time Exceeded" към подателя.
3. **Търсене в маршрутната таблица:** Рутерът анализира IP адреса на получателя (*Destination IP*) и търси съвпадение в своята маршрутна таблица.
4. **Longest Match Lookup:** Ако има няколко съвпадащи записа, се избира този с **най-дългата подмрежова маска** (най-специфичния път).
5. **Енкапсулация и препращане:** Преизчислява контролната сума на IP хедъра (заради променения TTL), капсулира пакета в нова Layer 2 рамка за съответния изходящ интерфейс и го изпраща.

3.3 Структура на маршрутна таблица (Routing Table)

Маршрутната таблица съдържа записи със следната базова структура:

- **Destination Network:** Адресът на целевата подмрежа.
- **Next Hop:** IP адресът на следващия рутер по пътя.
- **Metric (Метрика):** Стойност, указваща цената на пътя. Използва се за сравнение между маршрути, научени от един и същ протокол (по-ниската стойност е по-добра).

- **Administrative Distance (AD):** Степен на доверие към източника на маршрута (например: Директно свързан = 0, Статичен = 1, OSPF = 110, RIP = 120). Използва се при избор между различни маршрутизащи протоколи.

3.4 Маршрутизащи протоколи

- **Статично и динамично маршрутизиране** - статичните маршрути се задават ръчно от администратора; динамичните се изчисляват автоматично чрез протокол.
- **IGP (Interior Gateway Protocol) vs EGP (Exterior Gateway Protocol)** - IGP работи вътре в автономна система (RIP, OSPF), EGP - между автономни системи (BGP).

4. Протокол RIP (Routing Information Protocol)

4.1 Принципи на действие

RIP е **класически distance-vector** протокол, базиран на алгоритъма на **Белман-Форд**. Всеки рутер поддържа вектор от разстояния до всички познати мрежи и периодично го споделя с директно свързаните си съседи чрез UDP порт **520**.

Основното уравнение на distance-vector алгоритъма:

$$D_x(y) = \min_{v \in N(x)} \{c(x, v) + D_v(y)\}$$

където $D_x(y)$ е разстоянието, изчислено от рутер x до мрежа y , $N(x)$ са съседите на x , а $c(x, v)$ е цената на връзката до съсед v .

4.2 Метрика и ограничения

Метриката на RIP е броят на **хоповете** (междинните рутери) до мрежата на местоназначение. Максималната допустима стойност е **15**; стойност **16 се**

тълкува като "недостижимо". Това ограничение прави RIP **неподходящ за големи мрежи**, но опростява предотвратяването на безкрайни цикли.

4.3 Версии на RIP

	RIPv1	RIPv2
Адресиране	Classful (без маска)	Classless (поддържа CIDR/VLSM)
Изпращане на маска	Не изпраща подмрежова маска	Включва подмрежова маска
Разпространение	Broadcast (255.255.255.255)	Multicast (224.0.0.9)
Автентикация	Не	Да (обикновен текст/MD5)
Route tag	Не	Да

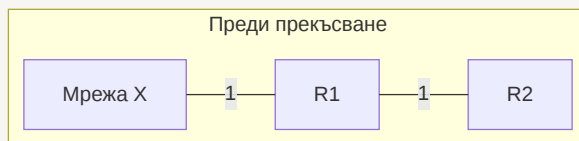
4.4 Таймери в RIP

- **Update timer** (~30 сек) - Контролира честотата на изпращане на периодичните актуализации.
- **Invalid/Expiration timer** (~180 сек) - Ако за даден маршрут не бъде получено обновяване в рамките на 180 секунди, той се обявява за невалиден и метриката му се установява на 16.
- **Holddown timer** (~180 сек) - Временно "замразява" маршрут след негативна промяна, за да не приеме преждевременно грешна информация.
- **Flush/Garbage collection timer** (~120 сек, след invalid) - Времето от последното валидно обновяване, след изтичането на което маршрутът се изтрива физически от таблицата (изпълнява се 60 секунди след Invalid таймера).

4.5 Проблеми с циклично маршрутизиране (Routing Loops) и решения

Тъй като всеки рутер вярва сляпо на съседите си, при прекъсване на връзка може да възникне маршрутизиращ цикъл, при който метриката бавно

нараства ("count to infinity"), докато достигне 16.



Пример: Ако връзката на R1 към мрежа X падне, но R2 все още рекламира стар маршрут "X: 2 хопа през R1", R1 може погрешно да приеме "X: 3 хопа през R2", после R2 приема "X: 4" и т.н., докато стойността "изтече" до 16.

Механизми за ограничаване на проблема:

- **Максимум 15 хопа** - Естествена горна граница на "броенето".
- **Split Horizon** - Рутерът няма право да изпраща информация за маршрут обратно през интерфейса, от който е научил този маршрут.
- **Poison Horizon** - Когато даден линк отпадне, рутерът незабавно анонсира този маршрут към интерфейса, от който го е научил, но с метрика 16 (∞). Това експлицитно указва на съседния рутер, че пътят е прекъснат, прекратявайки потенциални цикли.
- **Triggered updates** - при промяна маршрутът се разпространява веднага, а не се чака следващия редовен 30-секунден цикъл